

COURSE CODE : CSA1304

NAME:NIVETHA S

COURSE NAME : THEORY OF COMPUTATION

REG NO:192511269

LAB EXPERIMENTS

1. Write a C program to simulate a Deterministic Finite Automata (DFA) for the given language representing strings that start with a and end with a

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    char str[100];
```

```
    printf("Enter the string: ");
```

```
    scanf("%s", str);
```

```
    int len = strlen(str);
```

```
    if (len > 0 && str[0] == 'a' && str[len - 1] == 'a')
```

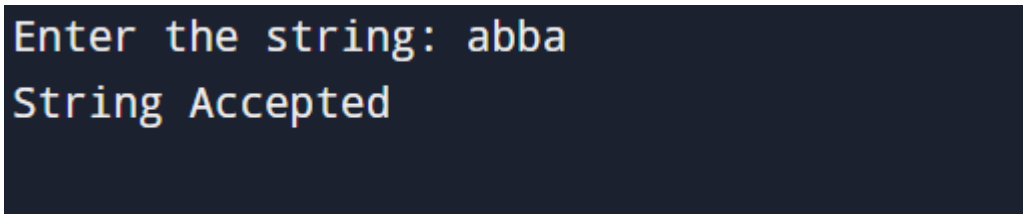
```
        printf("String Accepted\n");
```

```
    else
```

```
        printf("String Rejected\n");
```

```
    return 0;
```

```
}
```



```
Enter the string: abba
String Accepted
```

2. Write a C program to simulate a Deterministic Finite Automata (DFA) for the given language representing strings that start with 0 and end with 1

```
#include <stdio.h>
```

```
#include <string.h>
```

```

int main() {
    char str[100];

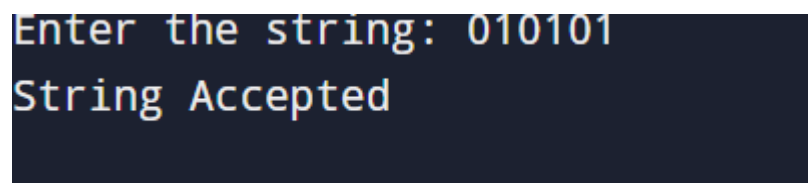
    printf("Enter the string: ");
    scanf("%s", str);

    int len = strlen(str);

    if (len > 0 && str[0] == '0' && str[len - 1] == '1')
        printf("String Accepted\n");
    else
        printf("String Rejected\n");

    return 0;
}

```



```

Enter the string: 010101
String Accepted

```

3. Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)

$$S \rightarrow 0A1 \quad A \rightarrow 0A \mid 1A \mid \epsilon$$

```
#include <stdio.h>
```

```
#include <string.h>
```

```

int main() {
    char str[100];
    int len;

    printf("Enter the string: ");
    scanf("%s", str);

```

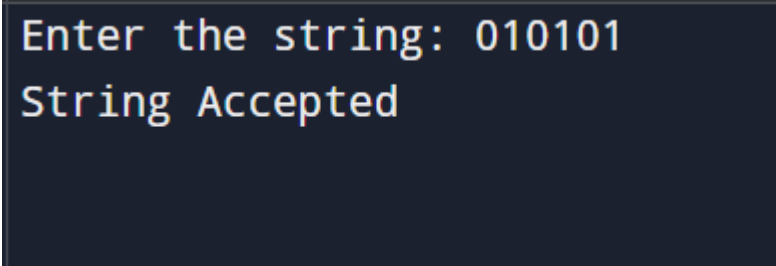
```

len = strlen(str);

if (len >= 2 && str[0] == '0' && str[len - 1] == '1')
    printf("String Accepted\n");
else
    printf("String Rejected\n");

return 0;
}

```



```

Enter the string: 010101
String Accepted

```

4. Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)

$S \rightarrow 0S0 \mid 1S1 \mid 0 \mid 1 \mid \epsilon$

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    char str[100];
```

```
    int i, len, flag = 1;
```

```
    printf("Enter the string: ");
```

```
    scanf("%s", str);
```

```
    len = strlen(str);
```

```
    for (i = 0; i < len / 2; i++) {
```

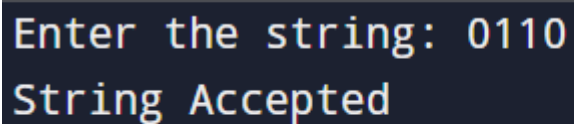
```

    if (str[i] != str[len - i - 1]) {
        flag = 0;
        break;
    }
}

if (flag)
    printf("String Accepted\n");
else
    printf("String Rejected\n");

return 0;
}

```



A terminal window with a dark background. The first line shows the prompt 'Enter the string: ' followed by the input '0110'. The second line shows the output 'String Accepted'.

5. Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)

$S \rightarrow OS0 \mid A A \rightarrow 1A \mid \epsilon$

```
#include <stdio.h>
```

```
#include <string.h>
```

```

int main() {
    char str[100];
    int len, i = 0, j, left0 = 0, right0 = 0;

    printf("Enter the string: ");
    scanf("%s", str);
}

```

```
len = strlen(str);
```

```
while (i < len && str[i] == '0') {  
    left0++;  
    i++;  
}
```

```
j = len - 1;  
while (j >= 0 && str[j] == '0') {  
    right0++;  
    j--;  
}
```

```
if (left0 != right0) {  
    printf("String Rejected\n");  
    return 0;  
}
```

```
for (i = left0; i <= j; i++) {  
    if (str[i] != '1') {  
        printf("String Rejected\n");  
        return 0;  
    }  
}
```

```
printf("String Accepted\n");  
return 0;  
}
```

```
Enter the string: 0011100
String Accepted

=== Code Execution Successful ===
```

6. Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)

$S \rightarrow 0S1 \mid \epsilon$

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    char str[100];
```

```
    int len, i, zeros = 0, ones = 0;
```

```
    printf("Enter the string: ");
```

```
    scanf("%s", str);
```

```
    len = strlen(str);
```

```
    i = 0;
```

```
    while (i < len && str[i] == '0') {
```

```
        zeros++;
```

```
        i++;
```

```
    }
```

```
    while (i < len && str[i] == '1') {
```

```
        ones++;
```

```
        i++;
```

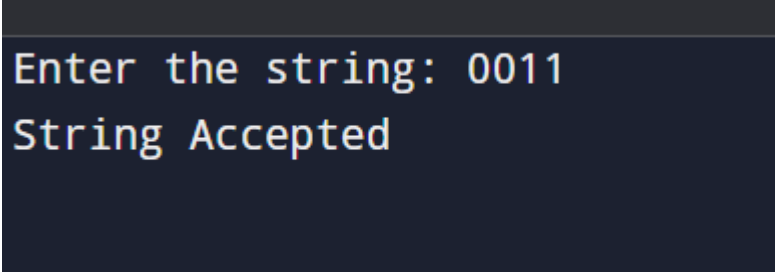
```
    }
```

```

    if (i == len && zeros == ones)
        printf("String Accepted\n");
    else
        printf("String Rejected\n");

    return 0;
}

```



```

Enter the string: 0011
String Accepted

```

7. Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)

$S \rightarrow A101A, A \rightarrow 0A \mid 1A \mid \epsilon$

```
#include <stdio.h>
```

```
#include <string.h>
```

```

int main() {
    char str[100];

    printf("Enter the string: ");
    scanf("%s", str);

    if (strstr(str, "101") != NULL)
        printf("String Accepted\n");
    else
        printf("String Rejected\n");

    return 0;
}

```

```
}
```

```
Enter the string: 1101010
```

```
String Accepted
```

```
=== Code Execution Successful ===
```

8. Write a C program to simulate a Non-Deterministic Finite Automata (NFA) for the given language representing strings that start with b and end with a

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main()
```

```
{
```

```
    char str[100];
```

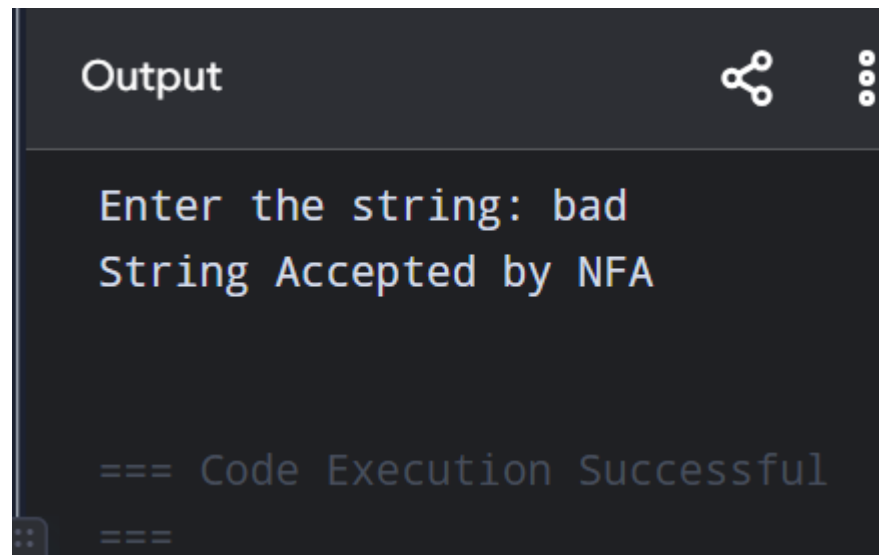
```
    printf("Enter the string: ");
```

```
    scanf("%s", str);
```



```
if (str[0] == 'b' && str[strlen(str) - 1] == 'd')
{
    printf("String Accepted by NFA\n");
}
else
{
    printf("String Rejected by NFA\n");
}

return 0;
}
```

A screenshot of a code execution environment's output window. The window has a dark background with a title bar at the top. The title bar contains the word "Output" on the left, a share icon in the center, and a vertical ellipsis icon on the right. The main area of the window displays the output of a program. It shows the prompt "Enter the string: bad" followed by the output "String Accepted by NFA". At the bottom, there is a status message "=== Code Execution Successful" followed by another "===" on the next line. A small icon with two dots is visible in the bottom-left corner of the window.

Output

Enter the string: bad
String Accepted by NFA

=== Code Execution Successful
===

9. Write a C program to simulate a Non-Deterministic Finite Automata (NFA) for the given language representing strings that start with 0 and end with 1

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main()
```

```
{
```

```
    char str[100];
```

```
    printf("Enter the string: ");
```

```
    scanf("%s", str);
```

```
    if (str[0] == '0' && str[strlen(str) - 1] == '1')
```

```
    {
```

```
        printf("String Accepted by NFA\n");
```

```
    }
```

```
    else
```

```
    {
```

```
        printf("String Rejected by NFA\n");
```

```
    }
```

```
    return 0;
```

```
Output  
```

```
Enter the string: 001
String Accepted by NFA

=== Code Execution Successful
===
```

10,11. Write a C program to find ϵ -closure for all the states in a Non-Deterministic Finite Automata (NFA) with ϵ -moves.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n, e;
```

```
    int i, j, k;
```

```
    int from, to;
```

```
    printf("Enter number of states: ");
```

```
    scanf("%d", &n);
```

```
    int epsilon[20][20] = {0};
```

```
    int closure[20][20] = {0};
```

```
    printf("Enter number of epsilon transitions: ");
```

```
    scanf("%d", &e);
```

```
    printf("Enter epsilon transitions (from to):\n");
```

```
for (i = 0; i < e; i++)
```

```
{
```

```
    scanf("%d %d", &from, &to);
```

```
    epsilon[from][to] = 1;
```

```
}
```

```
/* Every state belongs to its own epsilon closure */
```

```
for (i = 0; i < n; i++)
```

```
{
```

```
    closure[i][i] = 1;
```

```
}
```

```
/* Find epsilon closure */
```

```
for (k = 0; k < n; k++)
```

```
{
```

```
    for (i = 0; i < n; i++)
```

```
    {
```

```
        for (j = 0; j < n; j++)
```

```
        {
```

```
            if (closure[i][k] && epsilon[k][j])
```

```
            {
```

```
                closure[i][j] = 1;
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

```
/* Display epsilon closure */
```

```
printf("\nEpsilon Closure:\n");

for (i = 0; i < n; i++)
{
    printf("E-closure(q%d) = { ", i);

    for (j = 0; j < n; j++)
    {
        if (closure[i][j])
        {
            printf("q%d ", j);
        }
    }

    printf("}\n");
}

return 0;
}
```

Output



Enter number of states: 4

Enter number of epsilon
transitions: 3

Enter epsilon transitions
(from to):

0 1

1 2

2 3

Epsilon Closure:

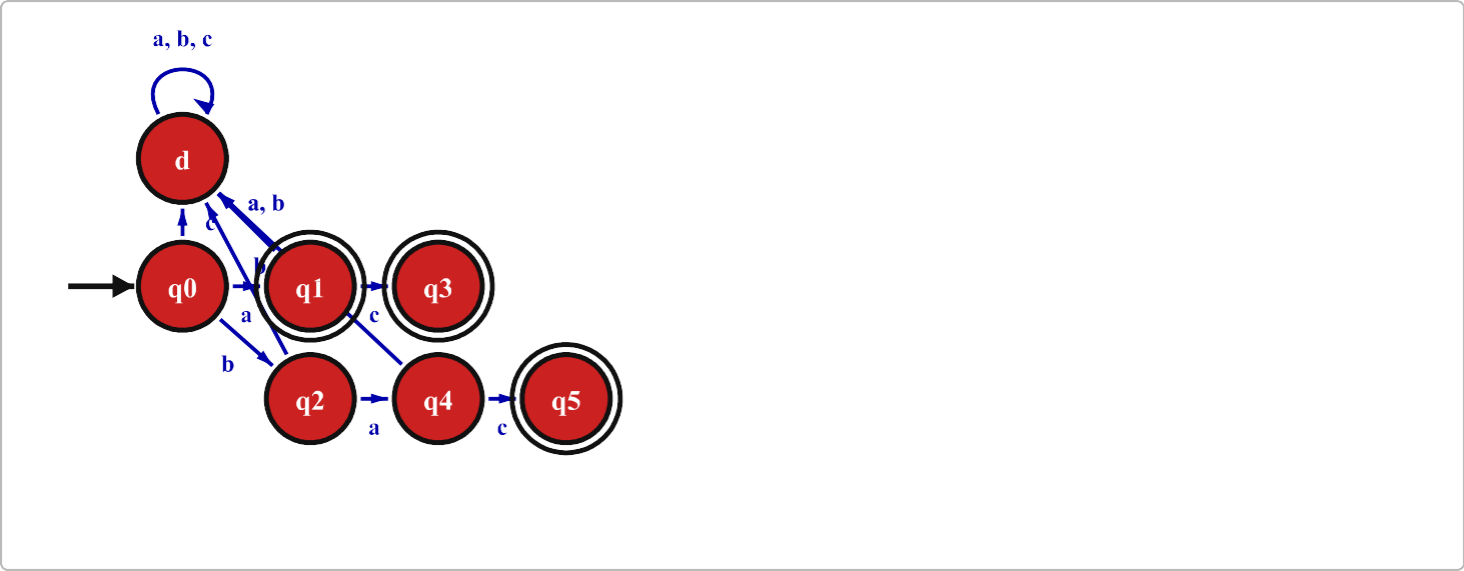
E-closure(q0) = { q0 q1 q2
q3 }

E-closure(q1) = { q1 q2 q3 }

E-closure(q2) = { q2 q3 }

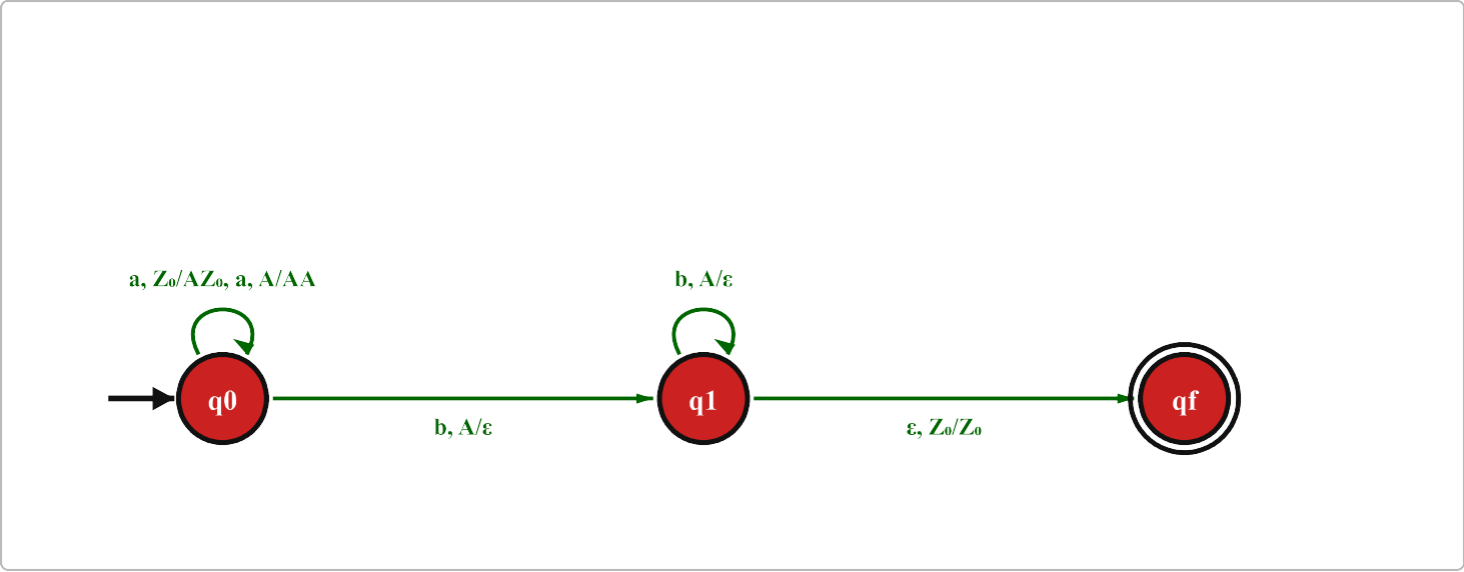
E-closure(q3) = { q3 }

12. DFA — DFA accepting "a", "ac", and "bac"



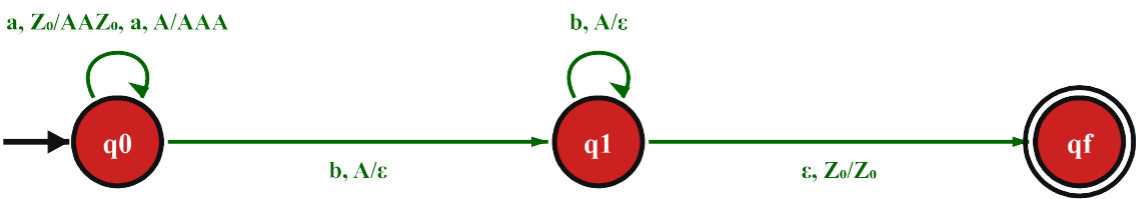
Type: DFA | States: $\{q_0, q_1, q_2, q_3, q_4, q_5, d\}$ | Start: q_0 | Accept: $\{q_1, q_3, q_5\}$

13. PDA — PDA accepting $a^n b^n$ ($n=2$)



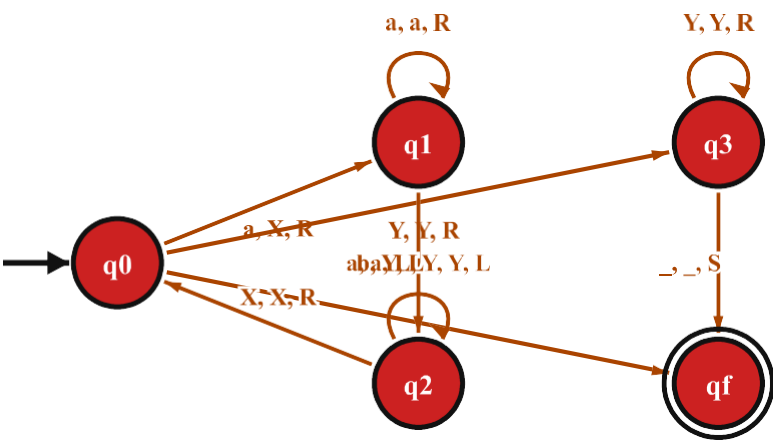
Type: PDA | States: $\{q_0, q_1, q_f\}$ | Start: q_0 | Accept: $\{q_f\}$

14. PDA — PDA accepting a^nb^{2n}



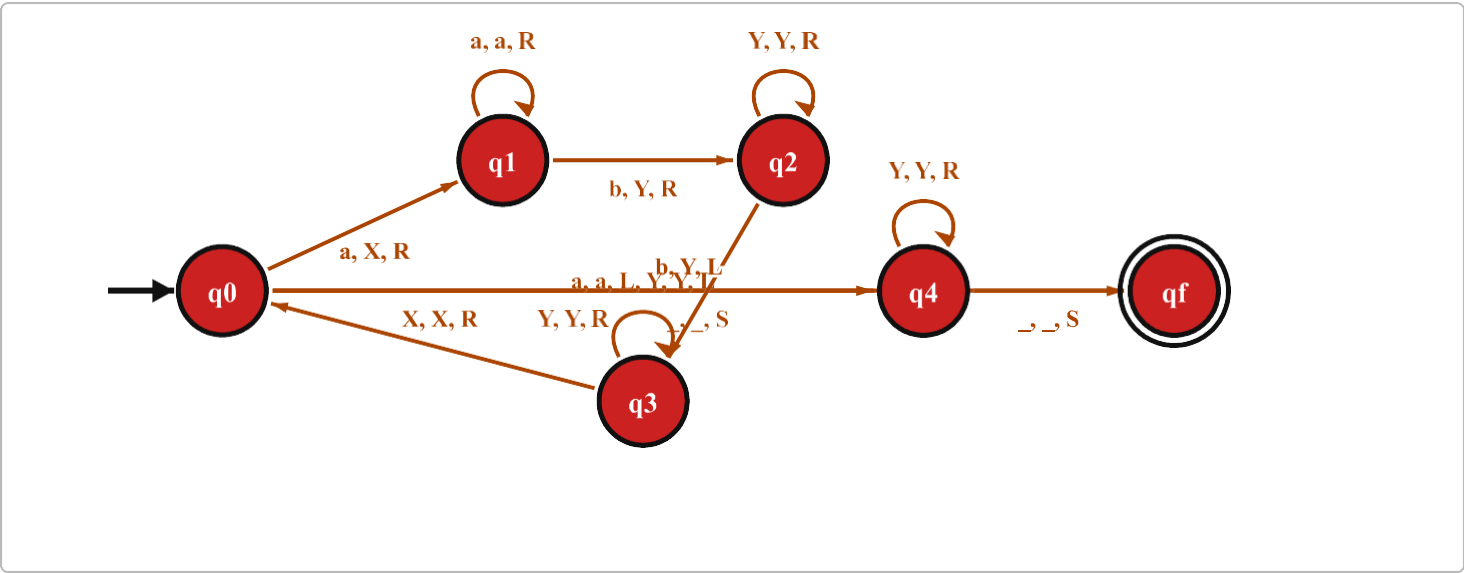
Type: PDA | States: { q_0, q_1, q_f } | Start: q_0 | Accept: { q_f }

15. TM — TM accepting a^nb^n



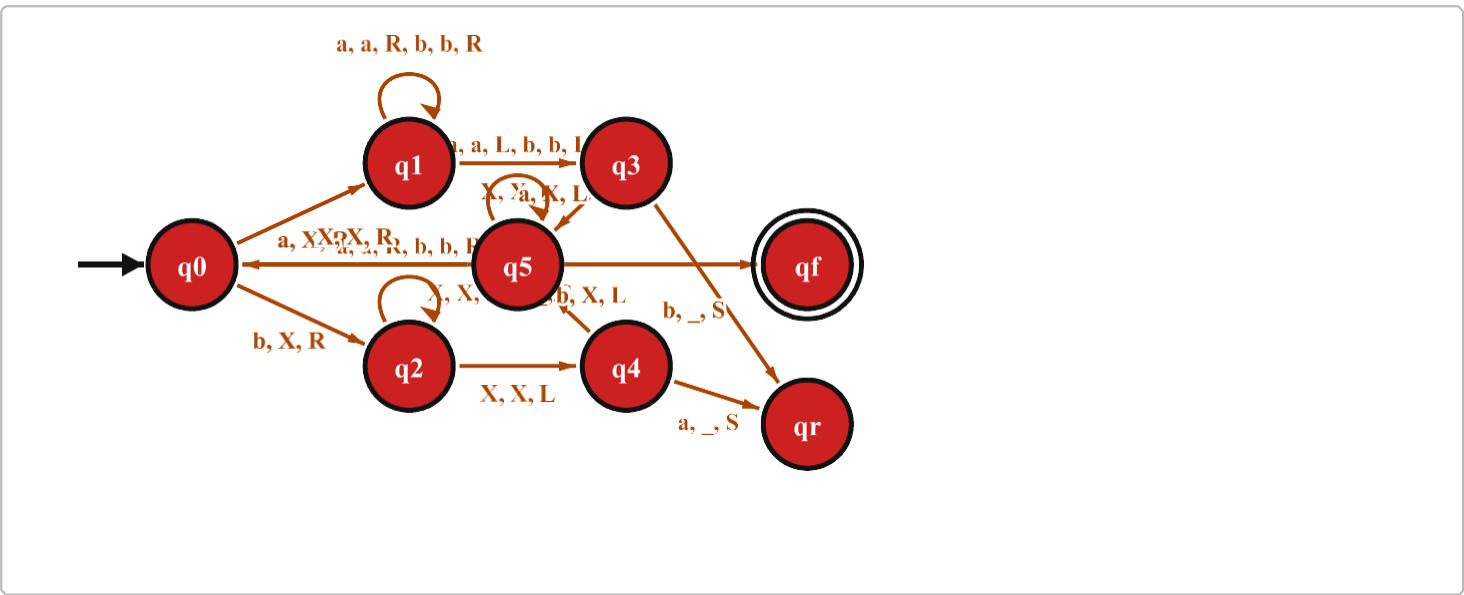
Type: TM | States: { q_0, q_1, q_2, q_3, q_f } | Start: q_0 | Accept: { q_f }

16. TM — TM accepting a^nb^{2n}



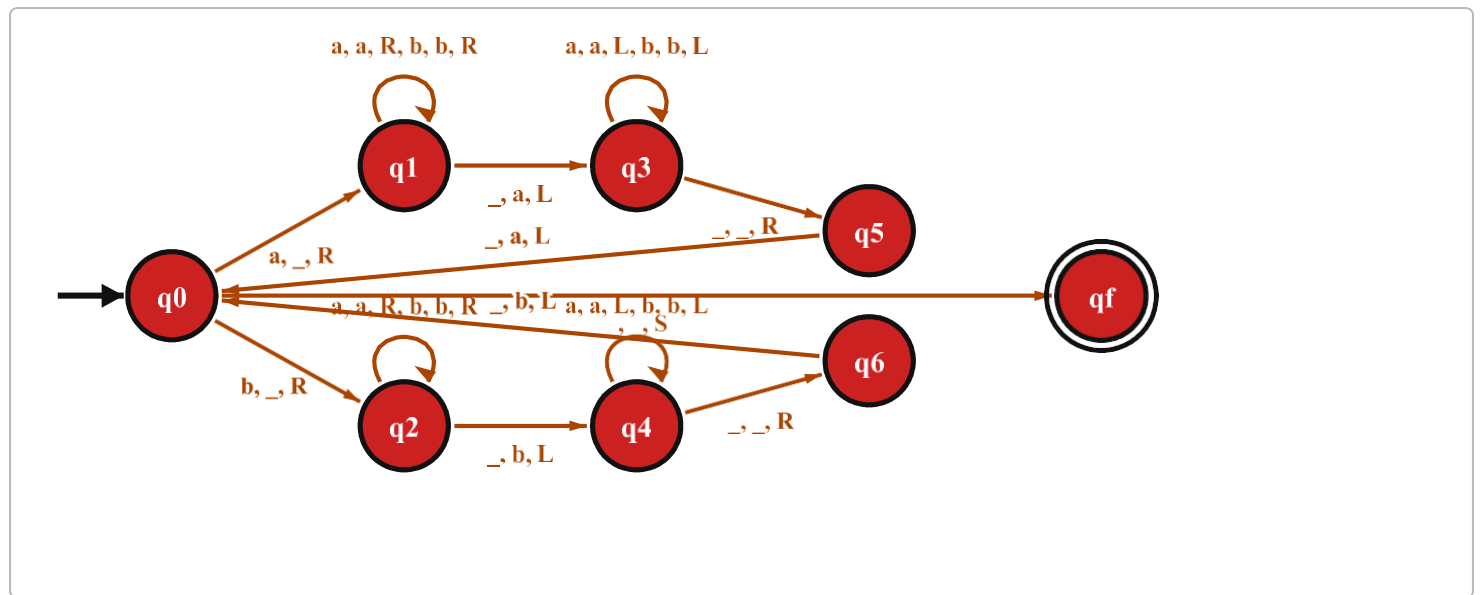
Type: TM | States: { $q_0, q_1, q_2, q_3, q_4, q_f$ } | Start: q_0 | Accept: { q_f }

17. TM — TM Palindrome "ababa"



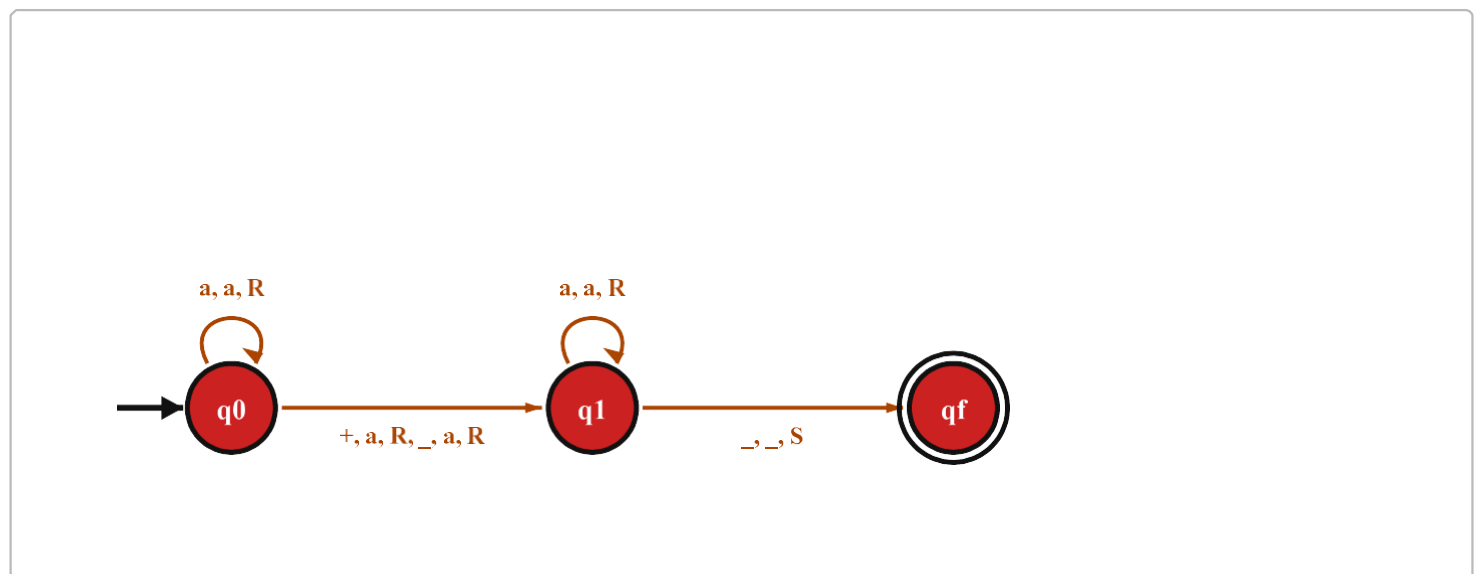
Type: TM | States: { $q_0, q_1, q_2, q_3, q_4, q_5, q_f, q_r$ } | Start: q_0 | Accept: { q_f }

18. TM — TM for ww (String Copying)



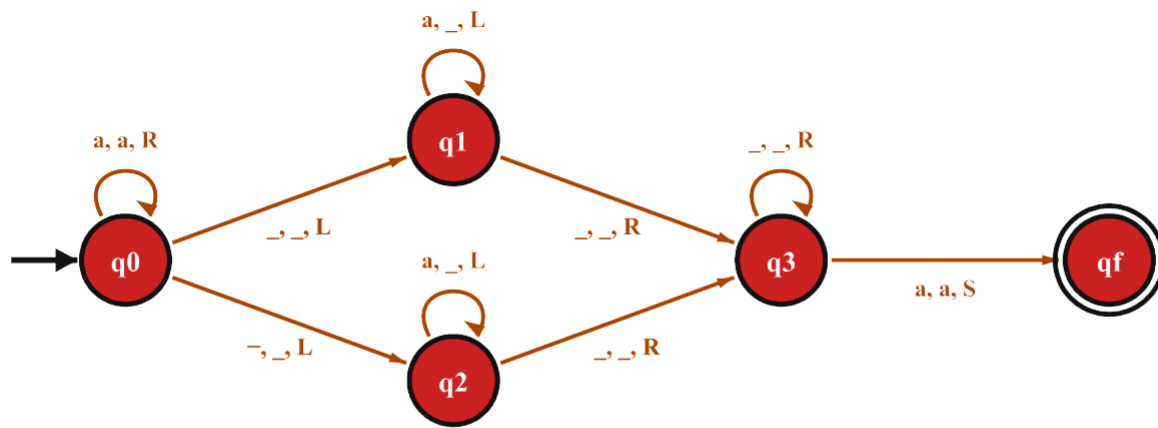
Type: TM | States: $\{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_f\}$ | Start: q_0 | Accept: $\{q_f\}$

19. TM — TM Addition "aa"+"aaa"



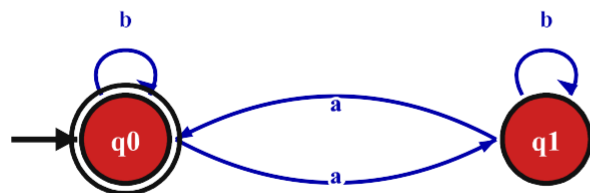
Type: TM | States: $\{q_0, q_1, q_f\}$ | Start: q_0 | Accept: $\{q_f\}$

20. TM — TM Subtraction "aaa"- "aa"



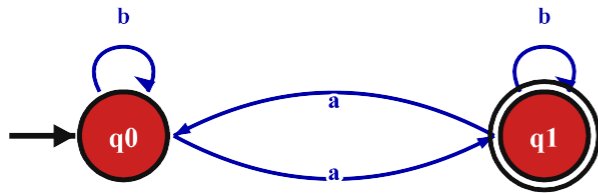
Type: TM | States: $\{q_0, q_1, q_2, q_3, q_f\}$ | Start: q_0 | Accept: $\{q_f\}$

21. DFA — DFA accepting even number of a's



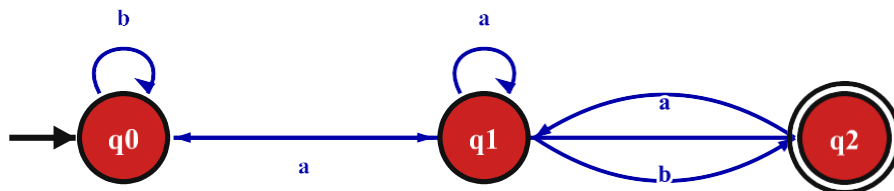
Type: DFA | States: $\{q_0, q_1\}$ | Start: q_0 | Accept: $\{q_0\}$

22. DFA — DFA accepting odd number of a's



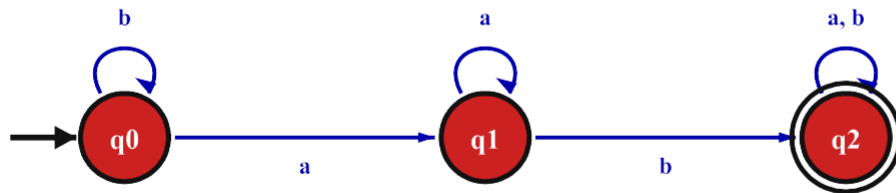
Type: DFA | States: $\{q_0, q_1\}$ | Start: q_0 | Accept: $\{q_1\}$

23. DFA — DFA ending with "ab" ($W = aaabab$)



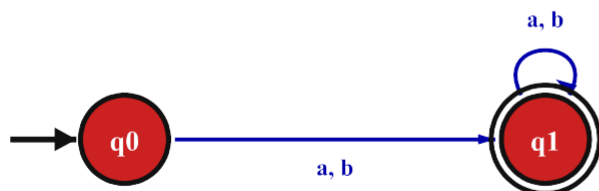
Type: DFA | States: $\{q_0, q_1, q_2\}$ | Start: q_0 | Accept: $\{q_2\}$

24. DFA — DFA having "ab" as substring



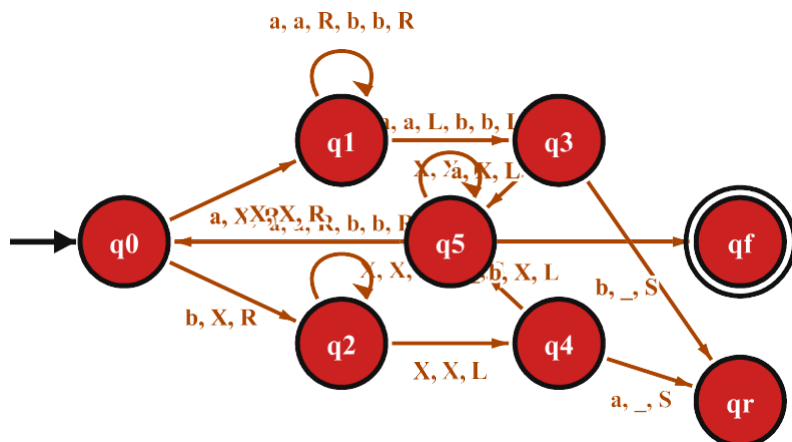
Type: DFA | States: $\{q_0, q_1, q_2\}$ | Start: q_0 | Accept: $\{q_2\}$

25. DFA — DFA starting with a or b



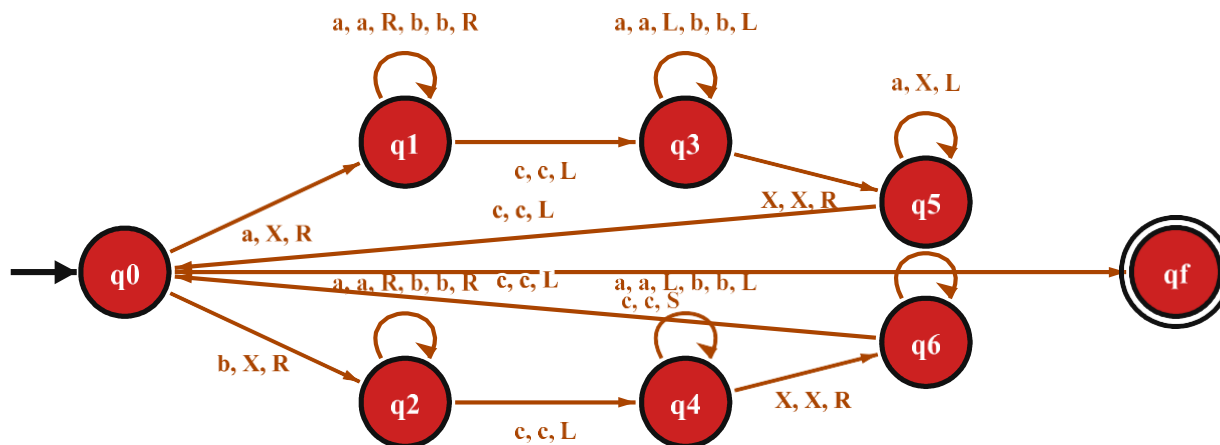
Type: DFA | States: $\{q_0, q_1\}$ | Start: q_0 | Accept: $\{q_1\}$

26. TM — TM Palindrome "bbabbb"



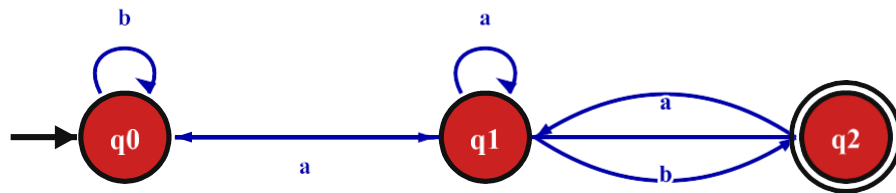
Type: TM | States: $\{q_0, q_1, q_2, q_3, q_4, q_5, q_f, q_r\}$ | Start: q_0 | Accept: $\{q_f\}$

27. TM — TM accepting wew



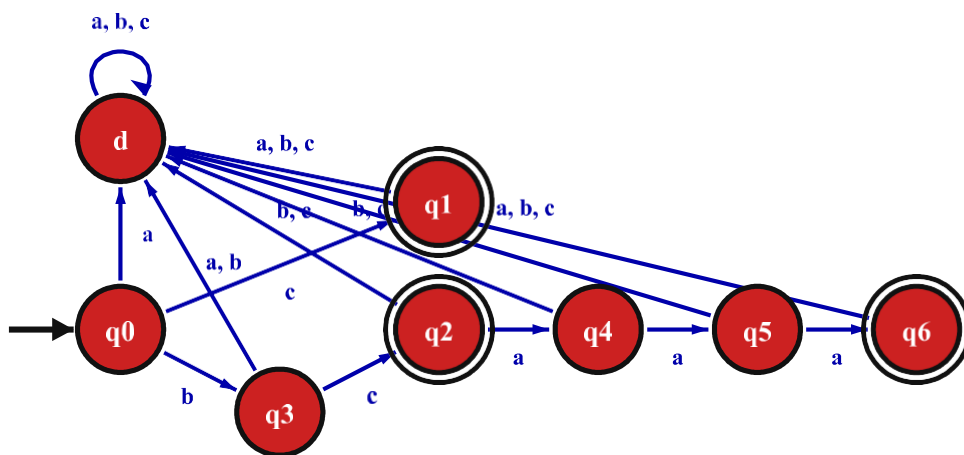
Type: TM | States: $\{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_f\}$ | Start: q_0 | Accept: $\{q_f\}$

28. DFA — DFA ending with "ab" (W = abbaabab)



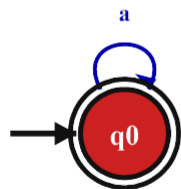
Type: DFA | States: $\{q_0, q_1, q_2\}$ | Start: q_0 | Accept: $\{q_2\}$

29. DFA — DFA accepting "bc", "c", and "bcaaaa"



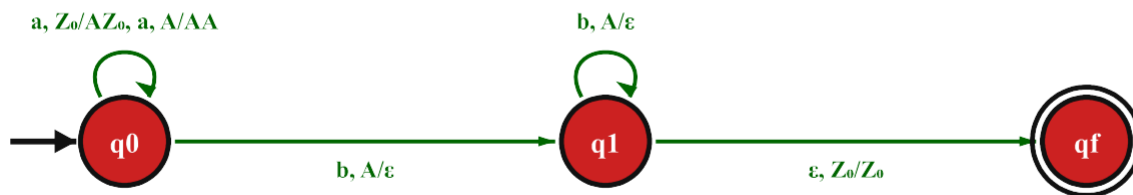
Type: DFA | States: $\{q_0, q_1, q_2, q_3, q_4, q_5, q_6, d\}$ | Start: q_0 | Accept: $\{q_1, q_2, q_6\}$

30. NFA — NFA accepting any number of a's



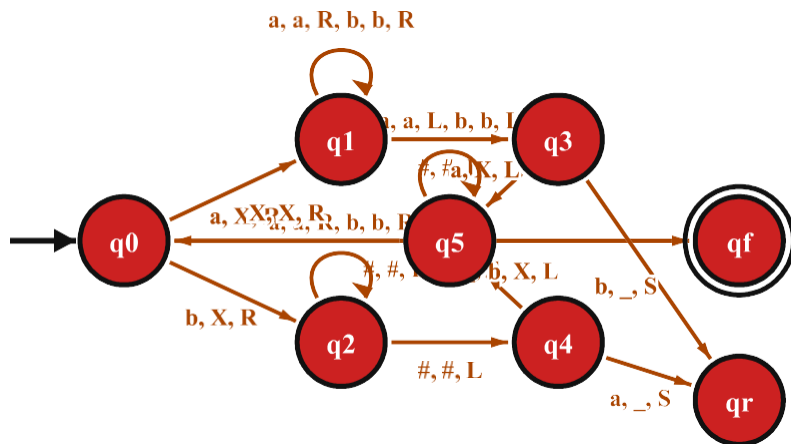
Type: NFA | States: $\{q_0\}$ | Start: q_0 | Accept: $\{q_0\}$

31. PDA — PDA accepting $a^n b^n$



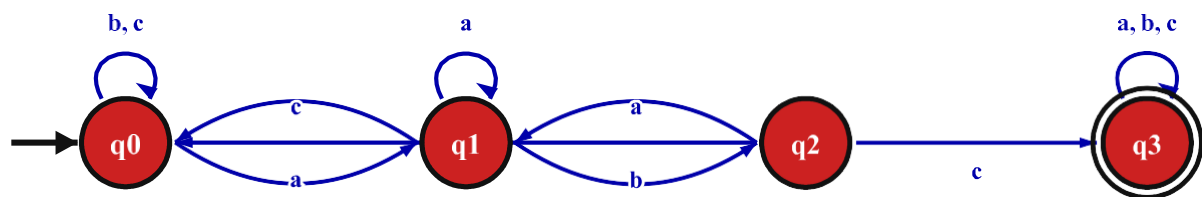
Type: PDA | States: $\{q_0, q_1, q_f\}$ | Start: q_0 | Accept: $\{q_f\}$

32. TM — TM String Comparison "aba"="aba"



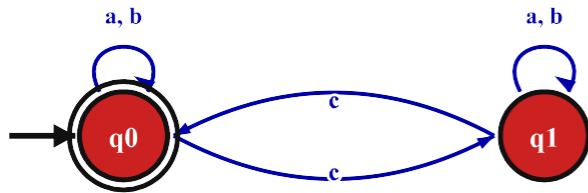
Type: TM | **States:** {q0,q1,q2,q3,q4,q5,qf,q_r} | **Start:** q0 | **Accept:** {qf}

33. DFA — DFA having "abc" as substring



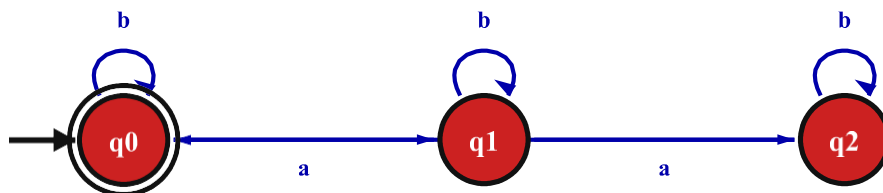
Type: DFA | **States:** $\{q_0, q_1, q_2, q_3\}$ | **Start:** q_0 | **Accept:** $\{q_3\}$

34. DFA — DFA accepting even number of c's



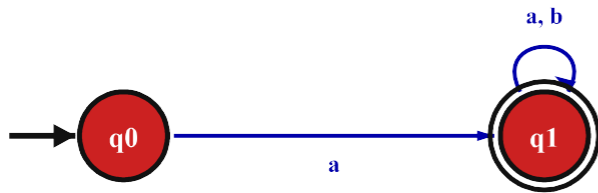
Type: DFA | States: $\{q_0, q_1\}$ | Start: q_0 | Accept: $\{q_0\}$

35. DFA — DFA where a's always appear tripled



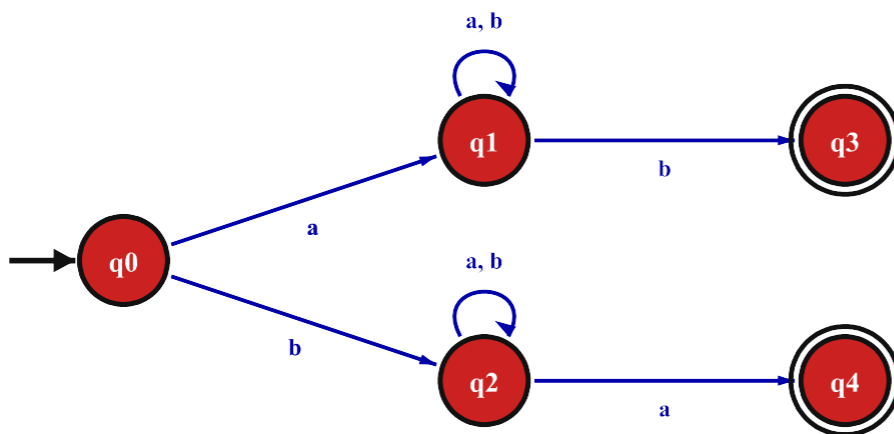
Type: DFA | States: $\{q_0, q_1, q_2\}$ | Start: q_0 | Accept: $\{q_0\}$

36. NFA — NFA start with a, end with b



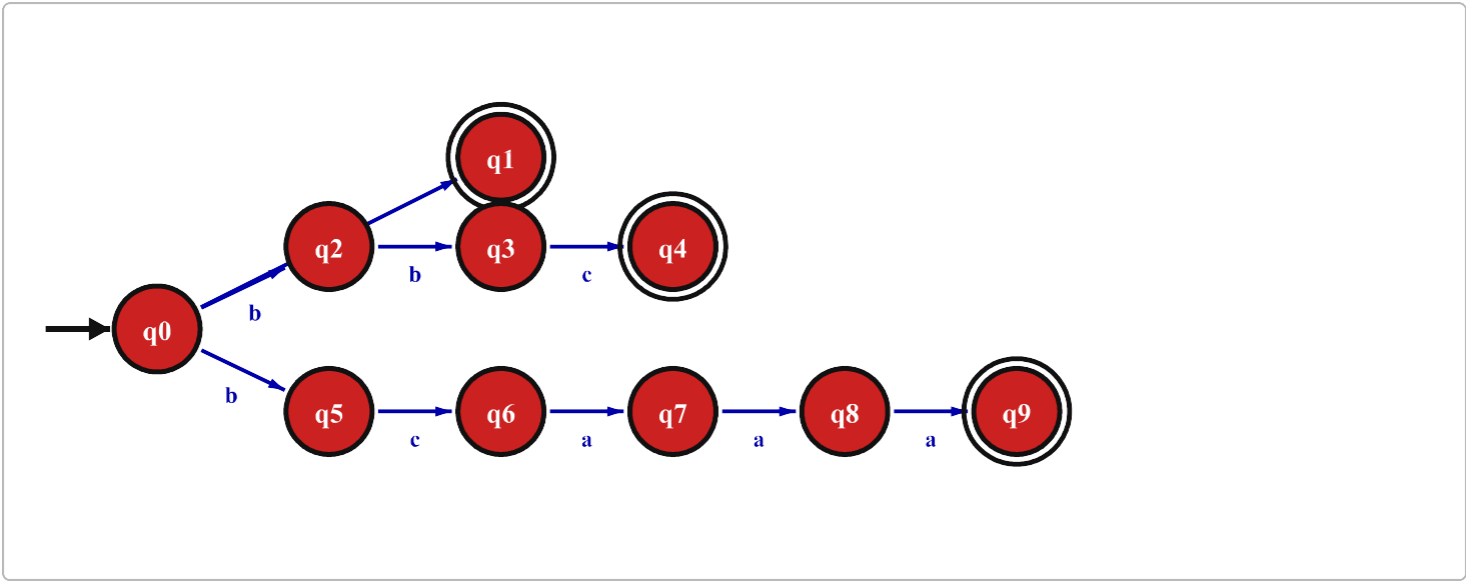
Type: NFA | States: $\{q_0, q_1\}$ | Start: q_0 | Accept: $\{q_1\}$

37. NFA — NFA start and end with different symbols



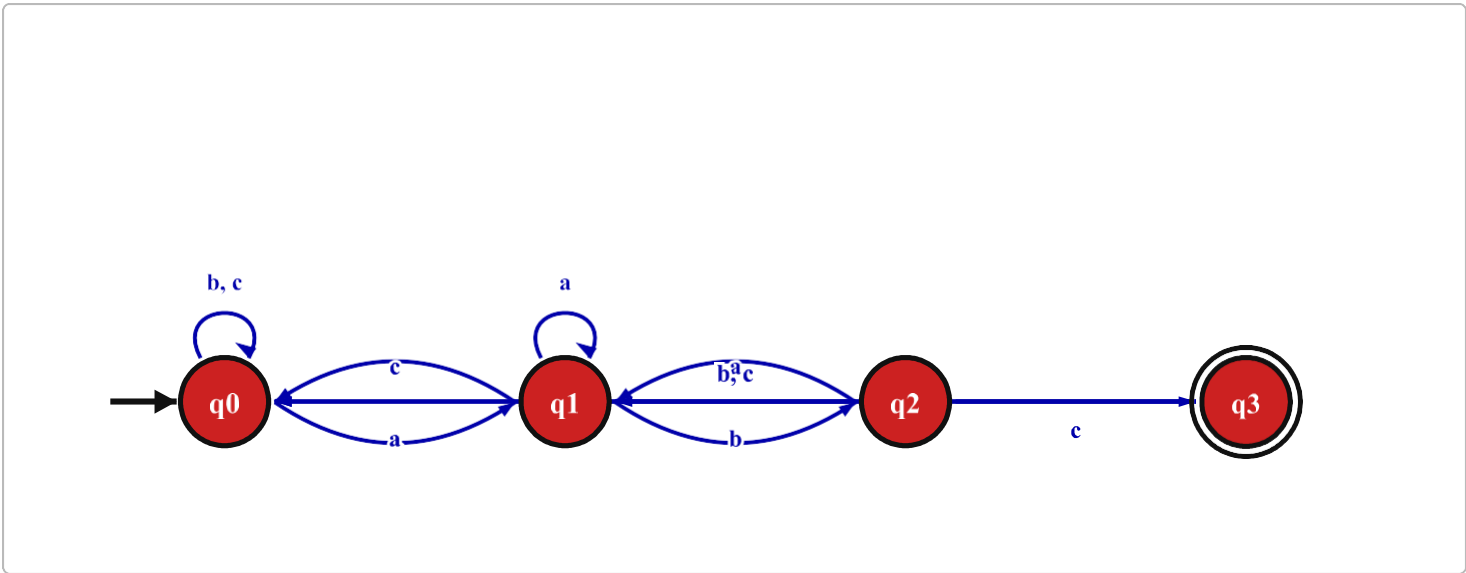
Type: NFA | States: $\{q_0, q_1, q_2, q_3, q_4\}$ | Start: q_0 | Accept: $\{q_3, q_4\}$

38. NFA — NFA accepting "bbc", "c", "bcaaa"



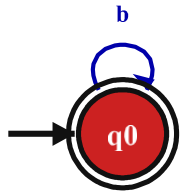
Type: NFA | States: {q0,q1..q9} | Start: q0 | Accept: {q1,q4,q9}

39. DFA — DFA ending with "abc" (W = abbaababc)



Type: DFA | States: {q0,q1,q2,q3} | Start: q0 | Accept: {q3}

40. NFA — NFA accepting any number of b's



Type: NFA | States: $\{q_0\}$ | Start: q_0 | Accept: $\{q_0\}$